國立臺北科技大學
National Taipei University of Technology

資料結構

Data Structure

Lab 08

姓名：簡紘田

學號：114310115

Lab08-Q1.Q2. (題目編號)
Briefly describe the problem. (題目簡述) Q1: Modify the inorder traversal function to a postorder traversal function inPart1. Q2: Implement a method to find the maximum value in the left and right subtrees.
Code

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

// 樹的節點
class TreeNode {
public:
    int value;              // 節點的值
    TreeNode* left;         // 左子節點
    TreeNode* right;        // 右子節點
    TreeNode(int val) : value(val), left(nullptr), right(nullptr) {} // 初始化節
點
};
// 樹結構
class BinaryTree {
public:
    TreeNode* root;
    BinaryTree() : root(nullptr) {} // 初始化樹

    // 用陣列構建二元樹
    TreeNode* buildTree(vector<int>& arr) {
        if (arr.empty()) return nullptr;

        queue<TreeNode*> q; // 儲存待處理的節點
        root = new TreeNode(arr[0]); // 建立根節點 (陣列第一個元素)
        q.push(root); // 將根節點加入 queue

        size_t i = 1; // 陣列索引
        while (!q.empty()) {
            TreeNode* current = q.front(); // 取出隊列中的節點
            q.pop();
```

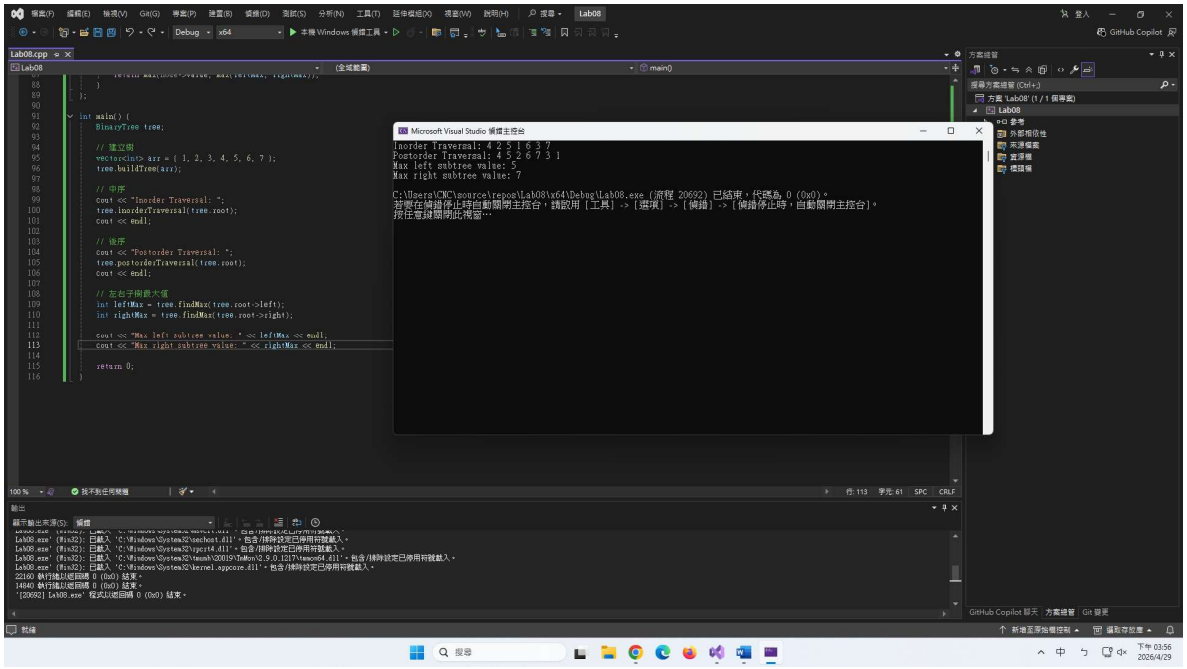
```
        // 添加左子節點
        if (i < arr.size()) {
            current->left = new TreeNode(arr[i]);
            q.push(current->left); // 將左子節點加入 queue
            i++;
        }
        // 添加右子節點
        if (i < arr.size()) {
            current->right = new TreeNode(arr[i]);
            q.push(current->right); // 將右子節點加入 queue
            i++;
        }
    }

    return root;
}
// 中序遍歷
void inorderTraversal(TreeNode* node) {
    if (node == nullptr) return; // 如果節點為空，忽略

    inorderTraversal(node->left);  // 遍歷左子樹
    cout << node->value << " ";    // 訪問當前節點
    inorderTraversal(node->right); // 遍歷右子樹
}

// 後序遍歷
void postorderTraversal(TreeNode* node) {
    if (node == nullptr) return; // 如果節點為空，忽略

    postorderTraversal(node->left); // 遍歷左子樹
    postorderTraversal(node->right); // 遍歷右子樹
    cout << node->value << " ";    // 訪問當前節點
}

// 找子樹最大值
int findMax(TreeNode* node) {
    // 空節點 → 回傳最小值
```

```
        if (node == nullptr) return INT_MIN;

        // 找左子樹最大
        int leftMax = findMax(node->left);

        // 找右子樹最大
        int rightMax = findMax(node->right);

        // 先假設目前節點最大
        int currentMax = node->value;

        // 跟左子樹比
        if (leftMax > currentMax) {
            currentMax = leftMax;
        }

        // 跟右子樹比
        if (rightMax > currentMax) {
            currentMax = rightMax;
        }

        // 回傳最大值
        return currentMax;
    }
};

int main() {
    BinaryTree tree; // 宣告二元樹

    // 輸入陣列用於構建樹，NULL 表示空子節點
    vector<int> arr = { 1, 2, 3, 4, 5, 6, 7 };

    tree.buildTree(arr); // 建立樹

    // 中序遍歷輸出
    cout << "Inorder Traversal: ";
    tree.inorderTraversal(tree.root);
    cout << endl;
```

```
    // 中序遍歷輸出
    cout << "Postorder Traversal: ";
    tree.postorderTraversal(tree.root);
    cout << endl;

    // 左右子樹最大值
    int leftMax = tree.findMax(tree.root->left);
    int rightMax = tree.findMax(tree.root->right);

    cout << "Max left subtree value: " << leftMax << endl;
    cout << "Max right subtree value: " << rightMax << endl;

    return 0;
}
```

Result

```
Inorder Traversal: 4 2 5 1 6 3 7
Postorder Traversal: 4 5 2 6 7 3 1
Max left subtree value: 5
Max right subtree value: 7

D:\紘田\大學\資料結構\Lab08\tree(Lab08)\x64\Debug\tree(Lab08).exe (流程 21796) 已結束，代碼為 0 (0x0)。
若要在偵錯停止時自動關閉主控台，請啟用 [工具] -> [選項] -> [偵錯] -> [偵錯停止時，自動關閉主控台]。
按任意鍵關閉此視窗…
```



Q3.Q4. Discussion

Q3: Explain how to find the maximum value in the left and right subtrees.

每個節點都會對左右與自己進行比大小，值都回傳空節點。

Q4: What happens if the left and right subtrees are not complete trees?

不會影響結果，因為是一個點一個點走的，跟樹的形狀沒有關係，如果遇到空的節點時，會回傳 INT_MIN，並不會影響比較結果。